

学号： 2106410115

# 大数据技术与基础 课后作业

学 生 姓 名： 李行健 专 业 班 级： 计算机 2101 班

五、抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

### 一、代码

```
from bs4 import BeautifulSoup
import sqlite3
import xlwt
import urllib.request,urllib.error
import re
def main():
    baseurl="https://movie.douban.com/top250?start="
    datalist=getData(baseurl)
    savepath="豆瓣电影 Top250.csv"
    saveData(datalist,savepath)
    #askURL("https://movie.douban.com/top250?start=0")
    findLink=re.compile(r'<a href="(.*?)">')#影片详情链接规则
    findImgSrc=re.compile(r'<img.*src="(.*?)"',re.S)#影片图片链接规则
    findTitle=re.compile(r'<span calss="title">(.*?)</span>')#片名
    findRating=re.compile(r'<span                                     class="rating_num"
property="v:average">(.*?)</span>')#评分
    findJudge=re.compile(r'<span>(\d*)人评价</span>')#人数
    findInq=re.compile(r'<span class="inq">(.*?)</span>')#概况
    findBd=re.compile(r'<p class="">(.*?)</p>',re.S)#内容

#爬取网页
def getData(baseurl):
    datalist=[]
    for i in range(0,10):#调用获取页面信息的函数 10 次
        url=baseurl + str(i*25)
        html=askURL(url)
        soup=BeautifulSoup(html,"html.parser")
        for item in soup.find_all('div',class_="item"):
            #print(item) #测试， 查看电影全部信息
            data=[]#保存一部电影的所有信息
            item=str(item)
            link=re.findall(findLink,item)[0]#影片详情链接
            data.append(link)

            imgSrc=re.findall(findImgSrc,item)[0]
            data.append(imgSrc)

            titles=re.findall(findTitle, item)
            if(len(titles)==2):
                ctitle=titles[0]
                data.append(ctitle)
                otitle=titles[1].replace("/","")#去掉无关的符号
                data.append(otitle)
```

```

else:
    data.append(titles)
    data.append(' ')#外国名留空
rating=re.findall(findRating, item)[0]
data.append(rating)

judgeNum=re.findall(findJudge, item)[0]
data.append(judgeNum)

inq=re.findall(findInq,item)
if len(inq)!=0:
    inq=inq[0].replace("。",",")
    data.append(inq)
else:
    data.append(" ")

bd=re.findall(findBd,item)[0]
bd=re.sub('<br(\s+)?/>(\s+)?'," ",bd)
bd=re.sub('/'," ",bd)
data.append(bd.strip())#去掉前后空格

datalist.append(data)#把处理好的一部信息存入 datalist
return datalist

#得到指定一个 URL 的网页内容
def askURL(url):
    head={}
    head={
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.101 Safari/537.36"}
    request=urllib.request.Request(url,headers=head)
    html=""
    try:
        response=urllib.request.urlopen(request)
        html=response.read().decode("utf-8")
        #print(html)
    except urllib.error.URLError as e:
        if hasattr(e,"code"):
            print(e.code)
        if hasattr(e,"reason"):
            print(e.reason)

    return html

def saveData(datalist,savepath):

```

```

    print("save....")
    book=xlwt.Workbook(encoding="utf-8",style_compression=0)#创建 workbook
对象
    sheet=book.add_sheet('豆瓣电影 Top250',cell_overwrite_ok=True) #创建工作
表
    col=("电影详情链接","图片链接","影片中文名","影片外文名","评分","评价
数","概况","相关信息")
    for i in range(0,8):
        sheet.write(0,i,col[i])#列名
    for i in range(0,250):
        print("第%d 条"%(i+1))
        data=datalist[i]
        for j in range(0,8):
            sheet.write(i+1,j,data[j])
    book.save(savepath)

if __name__=="__main__":

    main()
    print("爬取完毕")

```

## 二、程序说明

这段程序是用来爬取豆瓣电影 Top250 的数据的。

(1) 程序的主要功能包括：

①定义了一个函数 `main()`，用于控制整个程序的运行。

在 `main()` 函数中，使用循环和字符串拼接的方式生成每一页的 URL，并调用 `askURL()` 函数获取网页的 HTML 内容。

②在 `askURL()` 函数中，构建了请求头部信息，并通过 `urllib` 库发送请求，获取网页内容。

③使用正则表达式在网页内容中匹配需要的数据，如电影详情链接、图片链接、影片中文名、影片外文名、评分、评价数、概况和相关信息。

④将匹配到的数据存入一个二维列表 `datalist` 中并调用 `saveData()` 函数，将 `datalist` 中的数据保存到 Excel 文件中。

(2) 程序的运行流程如下：

①调用 `main()` 函数，开始执行程序。

`main()` 函数根据豆瓣电影 Top250 的页面结构和 URL 规律，循环生成每一页的 URL，并调用 `askURL()` 函数获取网页内容。

②`askURL()` 函数根据 URL 和请求头部信息构建请求对象，发送请求，获取网页内容。

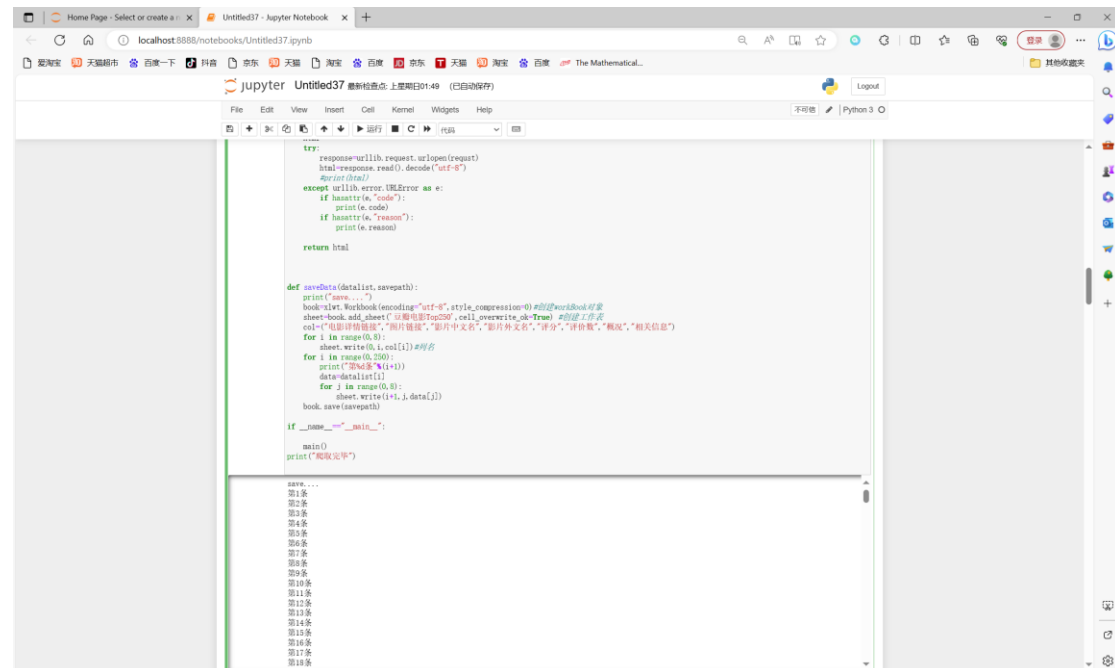
③使用正则表达式在网页内容中匹配需要的数据，并将匹配到的数据存入 `datalist` 中。

④循环结束后，调用 `saveData()` 函数，将 `datalist` 中的数据保存到 Excel 文件中。

⑤程序执行完毕，输出“爬取完毕”。

综上所述，主要实现了爬取豆瓣电影 Top250 的数据，并将数据保存到 Excel 文件中的功能。

### 三、运行截图

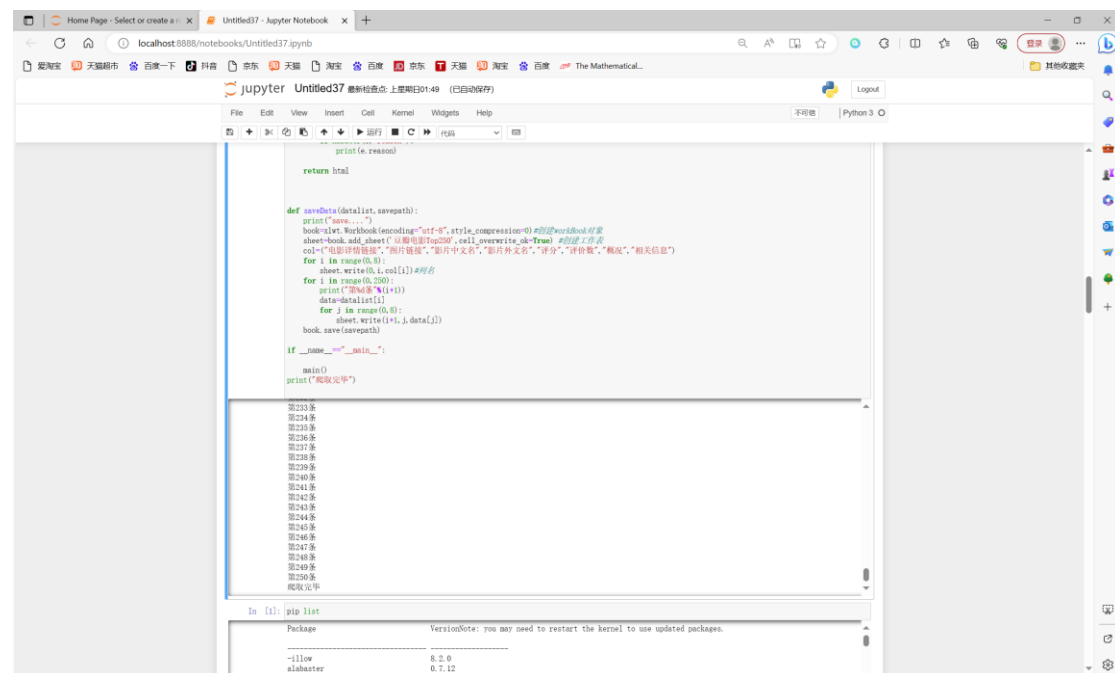


```
try:
    response=urlib.request.urlopen(request)
    html=response.read().decode("utf-8")
    #print(html)
except urllib.error.URLError as e:
    if hasattr(e,"code"):
        print(e.code)
    if hasattr(e,"reason"):
        print(e.reason)
    return html

def saveData(datalist,savepath):
    print("save...")
    book=xlwt.Workbook(encoding="utf-8",style_compression=0)#创建Workbook对象
    sheet=book.add_sheet('豆瓣电影Top250',cell_overwrite_ok=True)#创建工作簿
    col1="电影详细数据","影片链接","影片中文名","评分","评价数","概况","相关信息"
    for i in range(0,8):
        sheet.write(0,i,col1[i])#写头
    for i in range(0,250):
        print("第%d条"%(i+1))
        data=datalist[i]
        for j in range(0,8):
            sheet.write(i+1,j,data[j])
        book.save(savepath)

if __name__=="__main__":
    main()
print("爬取完毕")
```

第1条  
第2条  
第3条  
第4条  
第5条  
第6条  
第7条  
第8条  
第9条  
第10条  
第11条  
第12条  
第13条  
第14条  
第15条  
第16条  
第17条  
第18条



```
print(e.reason)

return html

def saveData(datalist,savepath):
    print("save...")
    book=xlwt.Workbook(encoding="utf-8",style_compression=0)#创建Workbook对象
    sheet=book.add_sheet('豆瓣电影Top250',cell_overwrite_ok=True)#创建工作簿
    col1="电影详细数据","影片链接","影片中文名","评分","评价数","概况","相关信息"
    for i in range(0,8):
        sheet.write(0,i,col1[i])#写头
    for i in range(0,250):
        print("第%d条"%(i+1))
        data=datalist[i]
        for j in range(0,8):
            sheet.write(i+1,j,data[j])
        book.save(savepath)

if __name__=="__main__":
    main()
print("爬取完毕")
```

第233条  
第234条  
第235条  
第236条  
第237条  
第238条  
第239条  
第240条  
第241条  
第242条  
第243条  
第244条  
第245条  
第246条  
第247条  
第248条  
第249条  
第250条  
爬取完毕

```
In [1]: pip list
```

| Package   | Version/Note: you may need to restart the kernel to use updated packages. |
|-----------|---------------------------------------------------------------------------|
| --llow    | 8.8.0                                                                     |
| alabaster | 0.7.12                                                                    |

[illegible]

六、 page 119 6.1

葡萄酒数据集 (wine.data) 搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

## 一、代码

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

## # 读取数据集

```
data = pd.read_csv('C:\\Users\\kobe\\Desktop\\wine.data', header=None)
```

## # 将数据集分为特征和标签

```
X = data.iloc[:, 1:]
```

```
y = data.iloc[:, 0]
```

```
from sklearn.preprocessing import LabelEncoder
```

#XGBoost 要求目标变量必须是从 0 开始的整数，而 y 变量取值是 [1, 2, 3]，使用 LabelEncoder 进行转换

```
le = LabelEncoder()
```

```
xgy = le.fit transform(y)
```

### # 将数据集分为训练集和测试集

[illegible]

```
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy,
test size=0.3, random state=42)
```

## # 建立决策树模型

```
dtc = DecisionTreeClassifier(random state=42)
```

```

dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)

# 建立随机森林模型
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)

# 建立 xgBoost 模型

xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)

# 输出各个模型在测试集上的表现
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)
请给我一下这个程序的说明

```

## 二、程序说明

- ①使用 pandas 库的 read\_csv 函数读取 wine 数据集，并将数据集分为特征和标签，其中特征为 X，标签为 y。
- ②使用 LabelEncoder 对标签 y 进行转换，使其从 0 开始的整数。
- ③使用 train\_test\_split 函数将数据集分为训练集和测试集，其中测试集占总数据集的 30%。
- ④建立决策树模型，使用 DecisionTreeClassifier 函数，并使用 fit 方法对训练集进行训练。
- ⑤使用 score 方法计算决策树模型在测试集上的准确率，并将结果保存在 dtc\_score 变量中。
- ⑥建立随机森林模型，使用 RandomForestClassifier 函数，并使用 fit 方法对训练集进行训练。除此之外，使用 score 方法计算随机森林模型在测试集上的准确率，并将结果保存在 rfc\_score 变量中。
- ⑦建立 XGBoost 模型，使用 XGBClassifier 函数，并使用 fit 方法对训练集进行训练。除此之外，使用 score 方法计算 XGBoost 模型在测试集上的准确率，并将结果保存在 xgb\_score 变量中。
- ⑧使用 print 函数输出各个模型在测试集上的表现，包括决策树模型的准确率、随机森林模型的准确率和 XGBoost 模型的准确率。

### 三、运行截图

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# 读取数据集
data = pd.read_csv('C:\\Users\\kobe\\Desktop\\wine.data', header=None)

# 将数据集分为特征和标签
X = data.iloc[:, 1:]
y = data.iloc[:, 0]
from sklearn.preprocessing import LabelEncoder
#XGBoost 要求目标变量必须是从 0 开始的整数, 而y变量取值是 [1, 2, 3], 使用LabelEncoder进行转换
le = LabelEncoder()
xgy = le.fit_transform(y)

# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy, test_size=0.3, random_state=42)
# 建立决策树模型
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)

# 建立随机森林模型
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)

# 建立xgBoost模型
xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)

# 输出各个模型在测试集上的表现
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)

Decision Tree Classifier score: 0.9629629629629629
Random Forest Classifier score: 1.0
XGBoost Classifier score: 0.9629629629629629
```